

Deep Convolutional Neural Networks (Lab)

Enrique Romero

Deep Learning
Master in Artificial Intelligence

Soft Computing Group
Computer Science Department
Universitat Politècnica de Catalunya, Barcelona, Spain

- 1 Analysis of Several Architectures
- 2 Example of Running a Training Script
- 3 Running your Scripts on MareNostrum (BSC)

- ① Open `Analysis_Architectures.ipynb`:
 - ② You will see an analysis of different architectures explained in the course:
 - LeNet-5
 - AlexNet
 - VGG16
 - GoogleNet
 - ResNet (ResNet34, ResNet50)
 - SqueezeNet (using Squeeze-and-Excitation blocks)
- including:
- Input and output sizes of the layers
 - Number and distribution of the parameters along layers
 - A final comparison of the number of parameters of every architecture

Take into account that:

- Torchvision has several of the explained architectures already implemented (AlexNet, VGG, GoogleNet, ResNet, SqueezeNet) and other ones that we have not not explained (DenseNet, MobileNet, EfficientNet,...)
- However, some of the implementations are slightly different from the original ones:
 - AlexNet: Some of the changes may be due to the fact that the network is not divided between two GPUs (see the comments in the script)
There is no “Local Response Normalization” (maybe removed as suggested in [Simonyan and Zisserman 2014]) and there is an “Adaptive Average-Pooling layer” (size: (6,6)) that is not present in the original paper
 - VGG16: There is an “Adaptive Average-Pooling layer” not present in the original paper (size: (7,7))

- A 2-D **Adaptive Average-Pooling layer** allows to obtain any desired output size by dividing the input into rectangular pooling regions, and computing the average of each region (while preserving the number of channels in the output)
- GoogleNet: The “Global Average Pooling” in GoogleNet can be implemented with an
AdaptiveAvgPool2d(output_size=(1, 1))

- 1 Analysis of Several Architectures
- 2 Example of Running a Training Script
- 3 Running your Scripts on MareNostrum (BSC)

Example of Running a Training Script

To run a script follow these steps:

- Open a terminal in your “local machine” (laptop,...)
- Change the working directory to `~/.../BSC-Scripts-Examples`
- Copy (via “`scp -pr FILES YourUsername@alogin1.bsc.es:DIR`”) the files/folders in BSC-Scripts-Examples:
 - dataset
 - MNIST-cnn-example-pytorch.ipynb
 - launcher-MNIST-cnn-example-pytorch.sh
- Login to BSC - Marenostum5:
`ssh YourUsername@alogin1.bsc.es`
- Change the working directory to the one where you have copied the files: “`cd DIR`”
- Run the script:
`sbatch -A nct_367 -q acc_training`
`launcher-MNIST-cnn-example-pytorch.sh`

More detailed instructions in the README-RUN file

- 1 Analysis of Several Architectures
- 2 Example of Running a Training Script
- 3 Running your Scripts on MareNostrum (BSC)

Running your Scripts on MareNostrum (BSC)

This course we have access to MareNostrum5 ACC (Accelerated Partition) at Barcelona Supercomputing Center (BSC)

- One account per student (unixgroup/account: nct_367)
- Connecting via *ssh* (no graphical interface)
- CPUs + GPUs computer cluster:
 - You are allowed to submit jobs
 - Your jobs will be run on any of the nodes in the cluster
- General information:
<https://bsc.es/supportkc/docs/MareNostrum5/intro>
<https://bsc.es/supportkc/docs/MareNostrum5/overview>
- All data will be lost at the end of the semester

First login:

- Login to the transfer1 machine (same username and password):

"ssh username@transfer1.bsc.es"

- Change your password:

"passwd"

Standard login:

"ssh username@alogin1.bsc.es"

See <https://bsc.es/supportkc/docs/MareNostrum5/logins>

Running your Scripts on MareNostrum (BSC)

In order to run a job, there are a number of steps to follow:

- 1) Create your (for example) python script as usual
("myscript.py")

```
import ...  
def myfunction(...):  
    ...  
    ...
```

- 2) Create a job script "launcher.sh"

```
#!/bin/bash  
#SBATCH ...  
module ...  
python myscript.py
```

- 3) Submit your (-A: account) job to a (-q) queue
"sbatch -A nct_367 -q acc_training launcher.sh"

Example:

- To run the “MNIST-cnn-example-pytorch.py” script:

```
sbatch -A nct_367 -q acc_training  
launcher-MNIST-cnn-example-pytorch.sh
```

After submitting a job, you can view the information of the scheduled jobs (running or waiting) with the command “*squeue*”

If you want to remove a job from the queue, run “*scancel {jobId}*”

Running your Scripts on MareNostrum (BSC)

launcher-MNIST-cnn-example-pytorch.sh:

```
#!/bin/bash
#SBATCH --qos=debug
#SBATCH --cpus-per-task=40
#SBATCH --gres gpu:1
#SBATCH --time=02:00:00
#SBATCH --job-name="test_mnist-pytorch-gpu"
#SBATCH --chdir=.
#SBATCH --output=test_mnist-pytorch-gpu_%j.out
#SBATCH --error=test_mnist-pytorch-gpu_%j.err
```

```
module purge
module load miniforge
module load cuda/12.6
source activate deepLearning
```

```
python MNIST-cnn-example-pytorch.py
```

Running your Scripts on MareNostrum (BSC)

Rest of basic commands (in addition to “*squeue*” and “*scancel*”):

See <https://bsc.es/supportkc/docs/MareNostrum5/slurm>

The accounts you have access can be obtained with the following command: “*source bsc_project list*”

The queues you have access (and the restrictions) can be obtained with the following command: “*bsc_queues*”

Running your Scripts on MareNostrum (BSC)

Job directives: The job script “launcher.sh” must include a set of directives to convey specific parameters of the job to the system, in the form “#SBATCH --directive=value”

Several commonly used directives (1):

- Request the “quality of service” (qos) for the job
#SBATCH --qos={qos}
- Time limit for the total runtime of the job
#SBATCH --time={time} [format DD-HH:MM:SS]
- Number of threads
#SBATCH --cpus-per-task={number}
- GPU specifications
#SBATCH --gres=gpu:{number}

Running your Scripts on MareNostrum (BSC)

Several commonly used directives (2):

- Set the job name
`#SBATCH --job-name=" myjobname"`
- Set the working directory
`#SBATCH --chdir={directory}`
- Redirect the standard output
`#SBATCH --output={filename}`
- Redirect the standard error
`#SBATCH --error={filename}`

And many more:

- <https://bsc.es/supportkc/docs/MareNostrum5/slurm>
- Run “*man sbatch*” for more options

Running your Scripts on MareNostrum (BSC)

Additionally, you must indicate the software you need for your script

This is implemented with an Environment Modules software, which enables the dynamic configuration of an environment through modules

Each module file incorporates the basic settings to configure the shell for running an application

More information at

<https://bsc.es/supportkc/docs/MareNostrum5/environment>

<http://modules.sourceforge.net>

Running your Scripts on MareNostrum (BSC)

Basic module commands:

- *"module h(elp)"*
- *"module av(ail)"* (displays available modules)
- *"module sp(ider)"* (displays available modules and extensions)
- *"module k(eyword) key1 key2 ..."* (searches for all possible modules matching any of the keys):
 - "module keyword tensorflow"*
 - "module keyword pytorch"*
- *"module av(ail)"* (displays all available modules)
- *"module pu(rge)"* (unloads all loaded modules)
- *"module lo(ad) module1 module2 ..."* (loads modules)
- *"module l(ist)"* (display all loaded modules)
- *"module unl(oad) module1 module2 ..."* (unloads modules)

Running your Scripts on MareNostrum (BSC)

There are also a number of conda environments already created, that you can use with *“module load ...”*

In order to search for a conda environment which contains a particular python package, run the command

```
“bsc_py_packages -N 99 -n package1 package2 ...”
```

For example, if you run

```
“bsc_py_packages -N 10 -n torchvision”
```

you will find 10 conda environments that you can load (with the *module* command) in order to run a script that uses the torchvision package

Running your Scripts on MareNostrum (BSC)

In our case, we have used an environment *“deepLearning”* inside the *“miniforge”* module where we can run our scripts:

“module purge”

“module load miniforge”

“source activate deepLearning”

Take into account that:

- There is no outside internet access
- Cluster jobs are executed in order, but they are queued depending on the resources requested, time limit, previous consumption, ...
- At infrequent times the system is less saturated
- It is not a good idea to wait until the last week to run the experiments

Good luck!